Les composants et les modèles de composants logiciels

Introduction

Les composants logiciels et leurs modèles constituent une approche fondamentale de l'architecture logicielle moderne, privilégiant **l'assemblage d'unités indépendantes** plutôt que l'écriture de blocs de code monolithiques.

Introduction

- L'architecture est « l'organisation fondamentale d'un système incorporée dans ses **composants**, les **relations entre ses composants** et leur **lien avec l'environnement** ainsi que les principes qui régissent sa conception et son évolution ». IEEE 42010

- Une architecture est la structure d'un système qui comprend des éléments logiciels appelés "**composants**" et des relations entre composants appelés "**connecteurs**".

Objet VS **Composant**

- En architecture logicielle, **objet** et **composant** sont deux notions liées mais situées à des niveaux d'abstraction différents.
- L'**objet** est **l'unité de construction du code**, tandis que le **composant** est **l'unité de construction du système**.
- On ne conçoit pas un composant sans objets (en POO), mais on peut créer des milliers d'objets sans jamais définir une architecture à base de composants.

Objet

- L'objet relève de la **Programmation Orientée Objet (POO)**.
- une instance d'une classe en POO.
- une **entité** qui **encapsule un état** (données) et **un comportement** (méthodes).
- Sa granularité est très fine, représentant généralement **une entité du monde réel** (un Utilisateur, une Facture).
- **Cycle de vie :** Il est créé en mémoire pendant l'exécution (via un new) et disparaît à la fin du programme
- Interagit avec d'autres objets via des appels de méthodes

Composant

- Le composant relève de l'**Architecture Logicielle**.
- unité logicielle indépendante, remplaçable et déployable qui remplit une fonction précise via une interface.
- Un composant contient souvent des dizaines d'objets.
- C'est une "boîte noire". On ne sait pas comment il est codé à l'intérieur, on sait juste ce qu'il fait (ex: un module de paiement, un service d'authentification).
- Il communique via des interfaces ou des contrats (API).
- Il est conçu pour être indépendant du langage. Un système peut être composé de briques hétérogènes communiquant via des API ou des standards de composants.

Organisation logique d'un composant

L'organisation logique d'un composant décrit **comment le composant est structuré en interne** et **comment il interagit avec le reste du système**, indépendamment des détails techniques d'implémentation.

Elle s'intéresse à **la structure conceptuelle et fonctionnelle** du composant afin de :

- **structurer le système**
- **faciliter la maintenance**
- **améliorer la réutilisation**
- **réduire le couplage entre composants**
- **clarifier les responsabilités**

Définition d'un composant logiciel

Un composant est une unité autonome dans le système.

Il offre des interfaces appelées **« interfaces fournies** » et en requiert d'autres appelées **« interfaces requises** ».

La notation UML du composant

Définition d'un composant logiciel

Interface

Le composant interagit avec son environnement de manière strictement contrôlée via des Ports et des Interfaces.

Il existe deux types d'interfaces :

✓ **Interface fournie** (représentée par un cercle et un trait plein)

regroupe l'ensemble des **services et opérations que le composant offre** à son environnement ou aux autres composants.

Elle définit les opérations que les autres peuvent appeler.

✓ **Interface requise** (représentée par un demi-cercle et un trait plein)

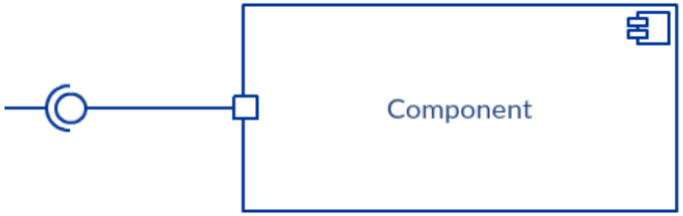
interface fournie par un autre composant

Elle définit les **services et ressources dont le composant a besoin** pour fonctionner et remplir ses propres tâches. Elle sert à rendre les **dépendances** du composant envers son contexte totalement explicites.

Définition d'un composant logiciel

Port

Le port (représenté par le petit carré à l'extrémité d'une interface requise ou d'une interface fournie) points d'interaction entre le composant et son environnement.



Dépendances

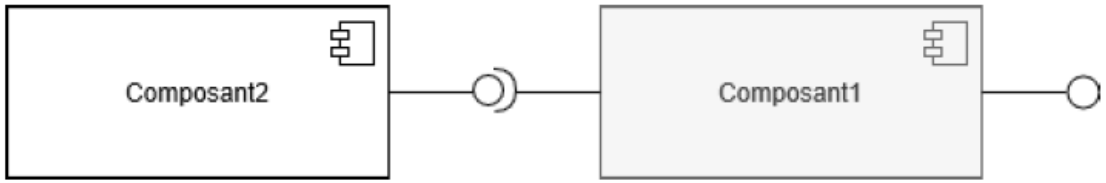
Une flèche de dépendance est utilisé pour montrer la relation entre deux composants.

Définition d'un composant logiciel

Connecteur

L'interaction entre deux composants est modélisée par un **connecteur**.

Il est modélisé par une paire **{interface requise, interface fournie}**.



La notation UML du connecteur

Une même interface de composant peut être invoquée selon plusieurs modes :

- Synchrone
- Asynchrone
- Invocation par Notification
- Invocation Différée

Définition d'un composant logiciel

Connecteur (suite)

Synchrone

- C'est le mode le plus simple et le plus courant.
- **Fonctionnement :** Le client appelle une méthode et suspend son exécution jusqu'à ce que le composant renvoie une réponse.
- **Usage :** Opérations rapides où le résultat est immédiatement nécessaire (ex: vérification de mot de passe).

Asynchrone

- Le client n'attend pas la fin de l'exécution pour continuer son travail.
- **Callback :** Le client fournit une fonction que le composant appellera une fois le travail terminé.
- **Future / Promise :** Le composant renvoie immédiatement un "objet promesse". Le client peut vérifier plus tard si le résultat est prêt.

  Usage : Traitements lourds ou appels

Invocation par Notification

- Le client envoie l'instruction mais ne s'intéresse pas au résultat, ni à savoir si l'exécution a réussi.
- **Fonctionnement** : L'interface ne définit souvent pas de valeur de retour (void).
- **Usage** : Logging, envoi de signaux dans des systèmes événementiels.

Invocation Différée

- Une variante où le client initie l'action, reçoit un identifiant de transaction, et revient "interroger" l'interface à intervalles réguliers pour savoir si le traitement est fini.
- **Usage** : Transferts de fichiers très longs ou processus métier humains.

Exemple:

Pour illustrer comment on découpe le logiciel en "blocs" logiques (les composants). On considère un **système de gestion de commandes pour un site e-commerce**.

Nous allons représenter le **composant GestionCommande**.

Ce dernier permet de créer une commande, annuler une commande, consulter une commande.

Le composant «**GstCommande»** contient :

Interface (services exposés)

- creerCommande()
- annulerCommande()
- consulterCommande()

Classes internes

- Commande
- CommandeService
- ValidationCommande

Dépendances

- composant **Paiement**
- composant **Stock**

Les principales caractéristiques d'un composant

Pour qu'un élément logiciel soit considéré comme un véritable "composant", il doit respecter plusieurs principes clés :

L'encapsulation Totale

- L'utilisateur ne sait pas comment il est codé à l'intérieur ; il sait seulement ce qu'il fait et comment interagir avec lui via ses interfaces.
- Ses détails d'implémentation sont masqués ; il est vu comme une « boîte noire ».

L'indépendance (Couplage Faible)

- Les composants doivent avoir le moins de dépendances possible entre eux. Si vous modifier un composant, cela ne doit pas influencer sur le reste du système.
- Il peut être déployé et remplacé sans impacter le reste du système, tant que l'interface est respectée.

La réutilisation

- Un composant est conçu pour être utilisé dans des contextes différents sans modification de son code source.
- Il est conçu pour être utilisé par des tiers dans différents contextes

Composant primitif et composite

La distinction entre eux repose principalement sur leur structure interne :

Le composant primitif :

- Il s'agit d'une unité qui ne contient pas d'autres composants. Il est "atomique" du point de vue de l'architecture.
- Il représente la brique de base fondamentale d'un système (il ne contient aucun sous-composant).
- Il possède des interfaces (offertes et requises) et une implémentation directe (du code source).
- C'est à ce niveau que réside l'implémentation réelle et le code source
- Il réalise une fonction métier précise et indivisible.

Le composant composite :

- Il s'agit d'une unité de composition plus complexe, construite à partir d'autres composants.
- Il peut regrouper aussi bien des composants primitifs que d'autres composants composites.
- Le composite agit comme un mécanisme permettant de gérer un groupe de composants comme un tout, tout en ayant la capacité de masquer certaines fonctionnalités de ses sous-composants.

L'aspect structurel et comportemental.

L'aspect structurel et **l'aspect comportemental** sont deux dimensions complémentaires et permettent de comprendre **comment un système est organisé** et **comment il fonctionne dans le temps**.

L'aspect structurel	• décrit **la structure interne et externe du composant** (définit la composition statique du composant)
L'aspect comportemental	• décrit **la manière dont le composant agit et réagit pendant l'exécution du système.** (comment le composant traite les requêtes et interagit avec les autres composants.)

L'aspect structurel et comportemental.

L'aspect structurel

- Principaux éléments structurels : Interfaces fournies, Interfaces requises, Ports, Sous-composants, connecteur.
- Questions : De quoi est composé le composant ? Quelles interfaces expose-t-il ? Quelles dépendances possède-t-il ?
- Représentation dans UML: Diagramme de composants, Diagramme de classes, Diagramme de packages

L'aspect comportemental

- Principaux éléments comportementaux : Séquence des opérations, Réactions aux événements, État interne du composant, Communication avec d'autres composants, Flux de messages
- Questions : Comment le composant exécute ses services ? Comment il réagit à une requête ? Quel est le flux d'interactions ?
- Représentation dans UML : Diagramme de séquence, Diagramme d'activités, Diagramme d'états

Cycle de vie d'un composant

Contrairement à un simple objet qui est créé et détruit manuellement par le programmeur, un composant est "géré" par un **conteneur** (comme Spring, EJB, ou un moteur de Microservices).

Le composant doit suivre un protocole strict pour que le système puisse le charger, l'utiliser et le supprimer proprement.

En architecture logicielle, on utilise souvent un **Diagramme d'États UML** pour visualiser ce cycle.

Cycle de vie d'un composant

L'Instanciation : Le conteneur lit la configuration et crée l'instance du composant en mémoire.

- **Action** : Le constructeur est appelé.
- **État** : Le composant existe, mais il n'est pas encore prêt

L'Initialisation: Le conteneur injecte les dépendances (les interfaces requises). Souvent appelée init()

- **Action :** Le conteneur connecte le composant aux autres services dont il a besoin.
- **État :** Le composant est **Prêt**.

L'Exécution : Le composant est maintenant opérationnel. Il reçoit des appels sur ses interfaces offertes et rend des services.

- **Action :** Traitement des requêtes métier.
- **État : Actif**.

La Destruction : Quand le système n'a plus besoin du composant, il doit libérer les ressources (fermer les connexions, les fichiers ou les ports réseau). Souvent appelée destroy()

- **Action :** Libération de la mémoire et des ressources.
- **État : Supprimé**.

L’approche par composants

C’est une méthode de conception dans le domaine de l’**architecture logicielle** qui consiste à construire un système logiciel à partir de **composants indépendants, réutilisables et interconnectés**. Cette approche vise à améliorer **la modularité, la réutilisabilité et la maintenance des systèmes logiciels**.

Cette approche permet de considérer le composant comme une **« boîte noire »** dont on ne connaît que les points d'interaction, facilitant ainsi la réutilisation et l'évolution du système,

Principe de l'approche par composants

Le principe fondamental est de **décomposer une application en composants logiciels autonomes**, chacun ayant une **fonction bien définie** et communiquant avec les autres via des **interfaces**.

Cette approche est largement utilisée dans les architectures modernes et peut être modélisée avec UML

Principe de l'approche par composants

L'approche par composant vise plusieurs objectifs importants :

Modularité : diviser le système en parties indépendantes. Chaque composant gère une responsabilité unique et clairement délimitée.

Réutilisabilité : utiliser les mêmes composants dans plusieurs applications.

Maintenabilité : faciliter la modification ou le remplacement d'un composant.

Évolutivité : permettre l'ajout de nouvelles fonctionnalités sans modifier tout le système.

Interoperabilité : permettre la communication entre composants différents.

Les modèles de composants

Un composant a besoin d'un **modèle de composants** (ou infrastructure) qui définit les règles du jeu.

Un **modèle de composant** définit les normes de construction, d'interaction et de composition des éléments. Pour fonctionner, ces composants nécessitent une **infrastructure** (ou plateforme) qui offre un **conteneur**.

Le Conteneur : C'est l'environnement d'exécution.

Il gère le cycle de vie du composant (création, exécution, destruction) et lui fournit des services techniques comme la sécurité, la gestion des transactions ou la persistance des données de manière transparente pour le développeur.

Les modèles de composants

Les principaux modèles du marché :

- **EJB (Enterprise Java Beans) :** Le standard pour Java EE. On y trouve des beans **Stateless** (sans état, partagés), **Stateful** (avec état par client) ou **Singleton** (instance unique).
- **CORBA (CCM) :** Modèle permettant l'intégration de composants hétérogènes sur des systèmes distribués.
- **Microsoft .NET / COM :** Utilisé principalement dans l'écosystème Windows.
- **UML :** Propose des diagrammes de composants pour modéliser l'architecture physique et les dépendances.

Les modèles de composants

Les modèles de composants industriels	Les modèles de composants académiques
• Ces modèles sont largement adoptés par le marché pour construire des applications d'entreprise robustes. Ils reposent généralement sur une infrastructure (conteneur) qui prend en charge les services techniques comme la sécurité, les transactions et la persistance	• Souvent issus de laboratoires de recherche, ces modèles explorent des concepts avancés comme la hiérarchie profonde, l'extensibilité et la séparation stricte entre contrôle et implémentation

Les modèles académiques **influencent directement** les standards industriels.
l'industrie pose les problèmes, l'académie formalise les solutions, et l'industrie les réabsorbe.

Les modèles de composants

Les modèles de composants industriels

CCM (CORBA Component Model)

- Common Object Request Broker Architecture (CORBA) est une norme définie par l'OMG. Elle repose sur deux éléments clés : ORB et IDL
- CORBA utilise l'IDL pour définir les services et l'ORB pour acheminer les appels entre objets distribués.
- CCM est un Standard inter-langages et inter-plateformes basé sur CORBA
- Les composants CCM sont gérés par un conteneur qui prend en charge : Transactions, Sécurité, Persistance, Cycle de vie et Communication entre composants

EJB (Enterprise JavaBeans)

- standard Java pour les applications d'entrepriseJava EEJakarta EE
- EJB permet au développeur de se concentrer sur la logique métier, tandis que le conteneur (JBoss, GlassFish) gère l'infrastructure.
- Le conteneur prend en charge les transactions, la sécurité, la persistance et le cycle de vie.

COM (Component Object Model)/DCOM (Distributed COM)/ .NET

- Les modèles de Microsoft permettant la communication entre objets, quel que soit le langage de programmation.
- DCOM : Extension de COM pour les systèmes distribués.
- En .NET, le composant est l'assembly : un fichier .dll ou .exe auto-décrit
- un assembly contient ses propres métadonnées qui décrivent sa version, sa culture, sa clé publique et toutes ses dépendances.
- .NET simplifie le développement de composants en fournissant un environnement d'exécution puissant et unifié.

Les modèles de composants

Les modèles de composants académiques

Fractal (INRIA) — Modèle récursif :

- un composant peut lui-même contenir d'autres composants (composites). Interfaces de contrôle séparées du comportement fonctionnel. Très influent en Europe.

Koala (Philips) —

- Conçu pour les systèmes embarqués. Distingue explicitement les interfaces *requises* et *fournies*, avec gestion fine des ressources contraintes.

SOFA (Université de Prague) —

- Composants avec ports, connecteurs et protocoles d'interaction formellement décrits. Vérification par model checking possible.

ACME / Wright (CMU)

- Langages de description d'architecture (ADL). ACME est un méta-langage pour décrire et échanger des architectures entre outils. Wright formalise les protocoles de communication entre composants.

UML et les composants

L'utilisation de l'**UML** (Unified Modeling Language) dans l'approche par composants est devenue un standard de fait pour documenter l'architecture logicielle. Bien qu'UML soit considéré comme une notation **semi-formelle** (contrairement aux ADL purement formels), il offre des outils puissants pour modéliser la structure et le comportement des systèmes à base de composants.

Les diagrammes clés pour l'approche par composants: **Diagramme de composants**, **Diagramme de structure composite**, **Diagramme de déploiement**, **Diagramme de paquetages.**

UML et les composants

1. Représentation du Composant et de ses Interfaces
   Un composant est représenté par un rectangle portant une icône spécifique en haut à droite.
   UML utilise la notation dite **« Lollipop »** (sucette) pour les **interfaces fournies** et la notation **« Socket »** (prise) pour les **interfaces requises.**
2. Points d'interaction : Les Ports
   les **ports** sont des points de connexion explicites situés sur la bordure du composant, Ils permettent de définir précisément par où passent les interactions avec l'environnement.
3. Mécanismes de connexion
   1. **Connecteur d'assemblage (Assembly Connector)**
      Il lie directement l'interface requise d'un composant à l'interface fournie d'un autre
   2. **Connecteur de délégation (Delegation Connector)**
      Dans le cas d'un composant **composite**, ce connecteur relie les ports du composant parent aux ports de ses composants internes (enfants), transmettant ainsi les appels vers l'intérieur ou l'extérieur du système

Avantages et inconvénients de l'approche par composant

Avantages

- **Réutilisabilité**: Un composant bien conçu peut être repris dans un autre projet sans modification.
- **Maintenabilité** : Un composant peut être modifié sans impacter tout le système (si les interfaces sont respectées).
- **Développement en parallèle** : chaque équipe peut travailler sur un composant différent
- **Testabilité** : On peut tester un composant de manière isolée avec des tests unitaires
- **Gain de productivité :** On assemble des composants existants au lieu de repartir de zéro.

Inconvénients

- **Complexité initiale :** Nécessite une bonne conception dès le départ.
- Conflits d'évolution (Versioning) :
- **Coût**: Coût élevé de sélection et d'intégration :
- **Évolutivité** : Risques de conflits de versions et incompatibilités.

En architecture, on dit souvent qu'**il n'y a pas de "solution parfaite", seulement des compromis (trade-offs)**.